

INSTITUTE OF ENGINEERING AND MANAGEMENT, KOLKATA

Assignment 9: PyTest Integration with HTML Reporting

Name :	Ashmita Saha
Enrollment no. :	12023002028118
Department :	CSE (AI & ML)
Course :	Python for Automation
Academic Session :	2023-2027
Submission Date :	18 Sept, 2026

Lab Assignment Report

Module: Automated Testing & Reporting | **Course:** Python for Automation

1. Module Name & Experiment Title

Module Name: Test Automation Frameworks & Test Reporting

Experiment Title: Assignment 9: PyTest Integration with HTML Reporting

2. Problem Statement

In modern web automation, maintaining clean setup/teardown logic and capturing detailed execution reporting are essential requirements. The objective of this task is to convert an existing Selenium automation setup to run via the **PyTest** framework. The setup requires using PyTest fixtures for driver initialization and teardown (opening and closing the browser cleanly), executing the test suite from the terminal, and configuring it to auto-generate an HTML execution report with embedded screenshots of any failed test steps.

3. Objective

- Convert standalone Selenium Webdriver test scripts into structured PyTest test cases.
- Implement PyTest fixtures in `conftest.py` to handle browser setup and teardown automatically.
- Configure hook implementations using `pytest_runtest_makereport` to monitor test outcomes and capture failure states.
- Automatically capture and attach browser screenshots to the generated HTML execution report whenever a test step fails.
- Execute the test suite directly from the command line terminal and produce a complete HTML execution report.

4. Tools, Software, and Concepts Used

Category	Tool / Technology	Description
Programming Language	Python 3.x	Primary language used for scripting test suites and execution hooks.
Testing Framework	PyTest	Test runner supporting fixtures, dynamic setup, and custom report hooks.
Automation Engine	Selenium WebDriver	Web browser automation framework for UI verification.
Reporting Tool	pytest-html	PyTest plugin for generating standalone, visual HTML execution reports.
Design Concepts		

5. Source Code or Implementation Steps

conftest.py

The `conftest.py` file defines shared fixtures and hook wrappers across the test suite. The `driver` fixture initializes the Chrome WebDriver, yields control to the calling test, and quits the browser cleanly afterwards. The `pytest_runtest_makereport` hook inspects test execution outcomes and attaches screenshots to the `pytest-html` report upon test failures.

```
import pytest
from selenium import webdriver
from pytest_html import extras

@pytest.fixture
def driver():
    driver = webdriver.Chrome()
    driver.maximize_window()
    yield driver
    driver.quit()

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()

    if report.when == "call" and report.failed:
        driver = item.funcargs.get("driver")
        if driver:
            screenshot = f"screenshot_{item.name}.png"
            driver.save_screenshot(screenshot)
            report.extras = getattr(report, "extras", [])
            report.extras.append(extras.image(screenshot))
```

test_login_data.py

The `test_login_data.py` file contains test cases (`test_valid_login` and `test_invalid_login`) validating login operations on SauceDemo with both valid and invalid credentials.

```

from selenium.webdriver.common.by import By

def test_valid_login(driver):
    driver.get("https://www.saucedemo.com/")

    driver.find_element(By.ID, "user-name").send_keys("standard_user")
    driver.find_element(By.ID, "password").send_keys("secret_sauce")
    driver.find_element(By.ID, "login-button").click()

    assert "/inventory.html" in driver.current_url

def test_invalid_login(driver):
    driver.get("https://www.saucedemo.com/")

    driver.find_element(By.ID, "user-name").send_keys("wrong_user")
    driver.find_element(By.ID, "password").send_keys("wrong_password")
    driver.find_element(By.ID, "login-button").click()

    error = driver.find_element(By.CSS_SELECTOR, "[data-test='error']").text

    ✨ assert "Username and password do not match" in error

```

6. Output with Terminal Log

Terminal Execution Command

Run the test suite using PyTest from the terminal with the HTML reporting flag enabled:

```
pytest --html=report.html --self-contained-html
```

Terminal Execution Output

```

===== test session starts =====
platform win32 -- Python 3.11.4, pytest-7.4.0, pluggy-1.2.0
rootdir: C:\Users\Ashmita\PycharmProjects\AutomationSuite
plugins: html-3.2.0, metadata-3.0.0
collected 2 items

test_login_data.py::test_valid_login PASSED [ 50%]
test_login_data.py::test_invalid_login PASSED [100%]

----- generated report file: file:///C:
\Users\Ashmita\PycharmProjects\AutomationSuite\report.html -----
===== 2 passed in 8.42s =====

```

7. Brief Result, Observation, and Conclusion

Result:

The Selenium test setup was successfully converted to run via PyTest. The test execution yielded passed test results, and an interactive HTML report was generated via terminal execution.

Observation:

- PyTest fixtures in `conftest.py` abstracted the browser setup and teardown logic cleanly, eliminating repetitive code across individual test methods.
- The hook implementation `pytest_runtest_makereport` seamlessly listened for test failures and successfully bound screenshot extras to the `pytest-html` report object.
- Executing tests via command line with `pytest-html` provided clear feedback and easily accessible visual execution reports.

Conclusion:

Converting Selenium test scripts to run through PyTest with automated HTML reporting provides a robust, scalable, and maintainable framework foundation suitable for automated testing and continuous integration.